

Conversational Agent Curriculum

Part of the [training document series](#). This folder is a self-contained curriculum for building a graph-based conversational agent — an LLM given tools and the autonomy to decide when to use them, typically calling an [MCP server](#) for its tools and surfaced through a [frontend](#) as a chat-like or report-like interface.

Reading order

1. **Core Concepts** — what "agent" means here, why a graph rather than a plain loop, nodes/edges/conditional routing, state schema and reducers, the tool-calling loop, and checkpoint-backed memory.
2. **Tool Calling, Guardrails and Generative UI** — using an MCP server as a tool source, classifying requests before acting on them, bounding the tool-calling loop, two different kinds of caching, streaming progress, mapping results to real UI components instead of freeform generated markup, and structural guardrails (required disclosures, capped model views, computed numbers, fixed follow-up lookups).
3. **Testing** — an injectable LLM seam for hermetic tests, stub-based graph-routing tests, testing reducers against a real graph invocation, real-subprocess integration testing of the tool connection, exactly one gated live-LLM smoke test, table-driven tests for the deterministic parts, and answer linting with a golden-prompt evaluation.
4. **Deployment** — trust boundaries between an agent and its tool source, session-id validation as a public input boundary, secrets, streaming responses, rate limiting, a safe runtime model-switch (admin) capability, and observability.

Prerequisites

Comfort with Python is assumed. Having read the [MCP Server curriculum](#) first is helpful but not required — this curriculum treats an MCP server as the typical source of an agent's tools, and several documents here point back to the equivalent MCP-side concept.

See also

- [Architecture Overview](#) for how this layer fits alongside a more conventional dashboard presentation layer
- [MCP Server](#) for the typical tool source this kind of agent calls
- [React Front End](#) for a typical UI shell a conversational agent is surfaced through